

这是初学者在求解“多数据输入”的题目时常范的错误，请读者留意。这种问题通常很隐蔽，但也不是发现不了：对于这个例子来说，编译时加一个-Wall 就会看到一条警告：warning: unused variable 's' [-Wunused-variable]（警告：没有用过的变量's'）。

提示 2-28：用编译选项-Wall 编译程序时，会给出很多（但不是所有）警告信息，以帮助程序员查错。但这并不能解决所有的问题：有些“错误”程序是合法的，只是这些动作不是所期望的。

2.5 注解与习题

不知不觉，本章已经开始出现一些挑战了。尽管难度不算太高，本章的例题和习题已经出现了真正的竞赛题目——仅使用简单变量和基本的顺序、分支与循环结构就可以解决很多问题。在继续前进之前，请认真总结，并且完成习题。

2.5.1 习题

习题 2-1 水仙花数 (daffodil)

输出 100~999 中的所有水仙花数。若 3 位数 ABC 满足 $ABC=A^3+B^3+C^3$ ，则称其为水仙花数。例如 $153=1^3+5^3+3^3$ ，所以 153 是水仙花数。

习题 2-2 韩信点兵 (hanxin)

相传韩信才智过人，从不直接清点自己军队的人数，只要让士兵先后以三人一排、五人一排、七人一排地变换队形，而他每次只掠一眼队伍的排尾就知道总人数了。输入包含多组数据，每组数据包含 3 个非负整数 a, b, c ，表示每种队形排尾的人数 ($a < 3, b < 5, c < 7$)，输出总人数的最小值（或报告无解）。已知总人数不小于 10，不超过 100。输入到文件结束为止。

样例输入：

```
2 1 6
2 1 3
```

样例输出：

Case 1: 41

Case 2: No answer

习题 2-3 倒三角形 (triangle)

输入正整数 $n \leq 20$ ，输出一个 n 层的倒三角形。例如， $n=5$ 时输出如下：

```
#####
#####
#####
###
#
```

习题 2-4 子序列的和 (subsequence)

输入两个正整数 $n < m < 10^6$, 输出 $\frac{1}{n^2} + \frac{1}{(n+1)^2} + \cdots + \frac{1}{m^2}$, 保留 5 位小数。输入包含多组数据, 结束标记为 $n=m=0$ 。提示: 本题有陷阱。

样例输入:

```
2 4
65536 655360
0 0
```

样例输出:

```
Case 1: 0.42361
Case 2: 0.00001
```

习题 2-5 分数化小数 (decimal)

输入正整数 a, b, c , 输出 a/b 的小数形式, 精确到小数点后 c 位。 $a, b \leq 10^6, c \leq 100$ 。输入包含多组数据, 结束标记为 $a=b=c=0$ 。

样例输入:

```
1 6 4
0 0 0
```

样例输出:

```
Case 1: 0.1667
```

习题 2-6 排列 (permutation)

用 1,2,3,...,9 组成 3 个三位数 abc , def 和 ghi , 每个数字恰好使用一次, 要求 $abc: def: ghi=1:2:3$ 。按照 “abc def ghi” 的格式输出所有解, 每行一个解。提示: 不必太动脑筋。下面是一些思考题。

题目 1。假设需要输出 2, 4, 6, 8, ..., $2n$, 每个一行, 能不能通过对程序 2-1 进行小小的改动来实现呢? 为了方便, 现把程序复制如下:

```
1 #include<stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d", &n);
6     for(int i = 1; i <= n; i++)
7         printf("%d\n", i);
8     return 0;
9 }
```

任务 1: 修改第 7 行, 不修改第 6 行。

任务 2: 修改第 6 行, 不修改第 7 行。

题目 2. 下面的程序运行结果是什么？“!=”运算符表示“不相等”。提示：请上机实验，不要凭主观感觉回答。

```
#include<stdio.h>
int main()
{
    double i;
    for(i = 0; i != 10; i += 0.1)
        printf("%.1f\n", i);
    return 0;
}
```

2.5.2 小结

循环的出现让程序逻辑复杂了许多。在很多情况下，仔细研究程序的执行流程能够很好地帮助理解算法，特别是“当前行”和变量的改变。有些变量是特别值得关注的，如计数器、累加器，以及“当前最小/最大值”这样的中间变量。很多时候，用 `printf` 输出一些关键的中间变量能有效地帮助读者了解程序执行过程、发现错误，就像本章中多次使用的一样。

别人的算法理解得再好，遇到问题时还是需要自己分析和设计。本章介绍了“伪代码”这一工具，并建议“不拘一格”地使用。伪代码是为了让思路更清晰，突出主要矛盾，而不是写“八股文”。

在程序慢慢复杂起来时，测试就显得相当重要了。本章后面的几个例题几乎个个都有陷阱：运算结果溢出、运算时间过长等。程序的运行时间并不是无法估计的，有时能用实验的方法猜测时间和规模之间的近似关系（其理论基础将在后面介绍），而海量数据的输入输出问题也可以通过文件得到缓解。尽管不同竞赛在读写方式上的规定不同，熟练掌握了重定向、`fopen` 和条件编译后，各种情况都能轻松应付。

再次强调：编程不是看书看会的，也不是听课听会的，而是练会的。本章后面的上机编程习题中包含了很多正文中没有提到的内容，对能力的提高很有好处。如有可能，请在上机实践时运用输出中间结果、设计伪代码、计时测试等方法。